

Lab Report No 5
Response of LTI Discrete Time System in Time Domain
Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted To:

Engg. Hareem Khan

Dated:

22nd March, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No A:

Solution:

Brief description (3-5 lines)

Zero-Input Response (Homogeneous Solution):

The zero-input response, also known as the homogeneous solution, represents the system's response when no input signal is applied. It is solely determined by the initial conditions of the system.

Mathematical Equation:

The zero-input response of an LTI system can be expressed using the system's difference equation and initial conditions. For a system represented by the

$$y_{zi}(n) = \sum_0^N a_k y_{zi}(n-k)$$

difference equation:

where a_k are the system output coefficients and $y(n-k)$ represents the system's output at time $n-k$.

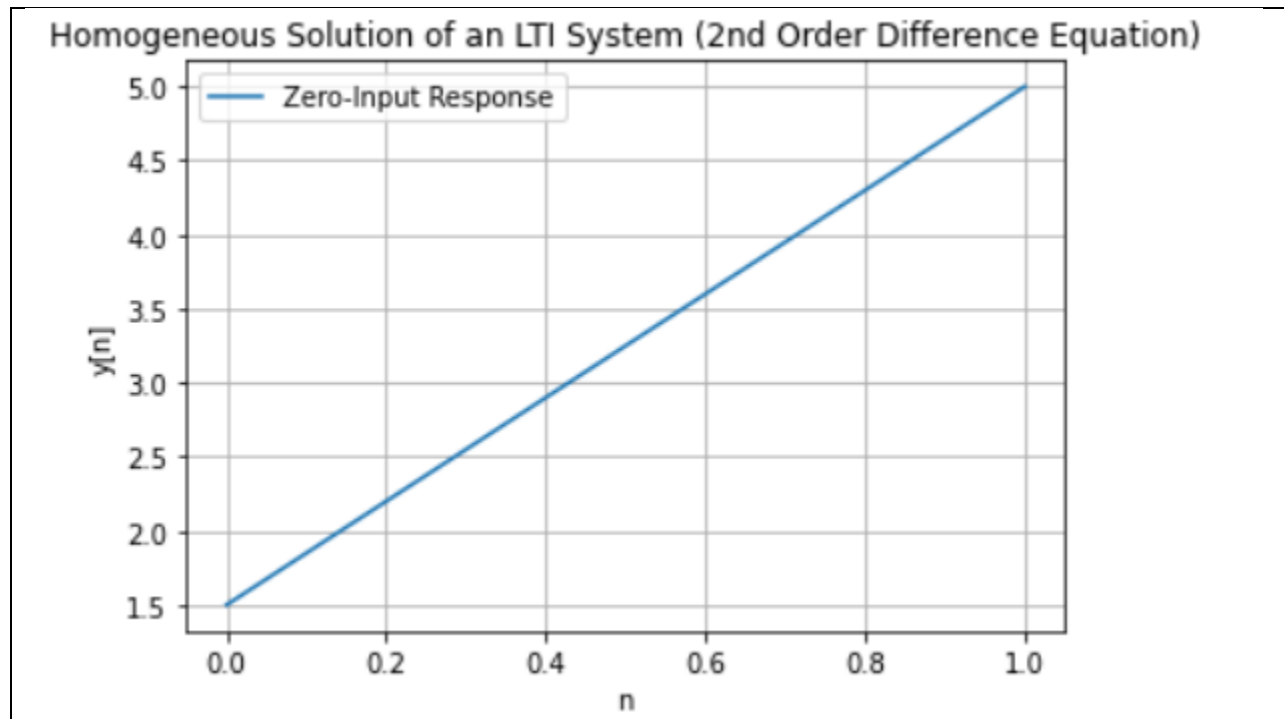
Example:

Consider the difference equation $y(n)=0.5y(n-1)+0.3y(n-2)$ with initial conditions $y(-1)=1$ and $y(-2)=2$. To find the zero-input response, we compute the system's response without any input signal applied.

The code

```
1 # Zero-Input Response (Homogeneous Solution)
2 import numpy as np
3 import matplotlib.pyplot as plt
4 a1, a2 = 0.5, 0.3
5 ic = [1, 2]
6 def zero_input_response(a1, a2, ic):
7     if len(ic) < 2:
8         print("At least 2 initial conditions are required for a 2nd order system.")
9         print("Adding a zero-valued initial condition to make the length 2.")
10        ic += [0]
11        a = [1, -a1, a2]
12        zi_response = [sum(ai * ic[m] for m, ai in enumerate(ai_array)) for ai_array in zip(a, ic[::-1])]
13        return zi_response[::-1]
14    zi_response = zero_input_response(a1, a2, ic)
15    n = np.arange(len(zi_response))
16    plt.plot(n, zi_response, label='Zero-Input Response')
17    plt.xlabel('n')
18    plt.ylabel('y[n]')
19    plt.title('Homogeneous Solution of an LTI System (2nd Order Difference Equation)')
20    plt.legend()
21    plt.grid()
22    plt.show()
```

The results (Screenshot)



Lab Task No B:

Solution:

Brief description (3-5 lines)

Particular Solution:

The particular solution represents the system's response to a specific input signal, ignoring the initial conditions. It captures the transient behavior of the system caused by the input signal.

Mathematical Equation:

The particular solution of an LTI system can be obtained by simulating the system with the given input signal.

$$y_p(n) = \text{System Simulation with Input Signal } x(n)$$

Example:

For the same difference equation $y(n) = x(n) + 0.3x(n-1) + 0.5x(n-2)$, suppose the input signal is a unit step function $x(n) = u(n)$. We simulate the system with this input signal to find the particular solution.

The code

```
# Particular Solution:
import numpy as np
import matplotlib.pyplot as plt
```

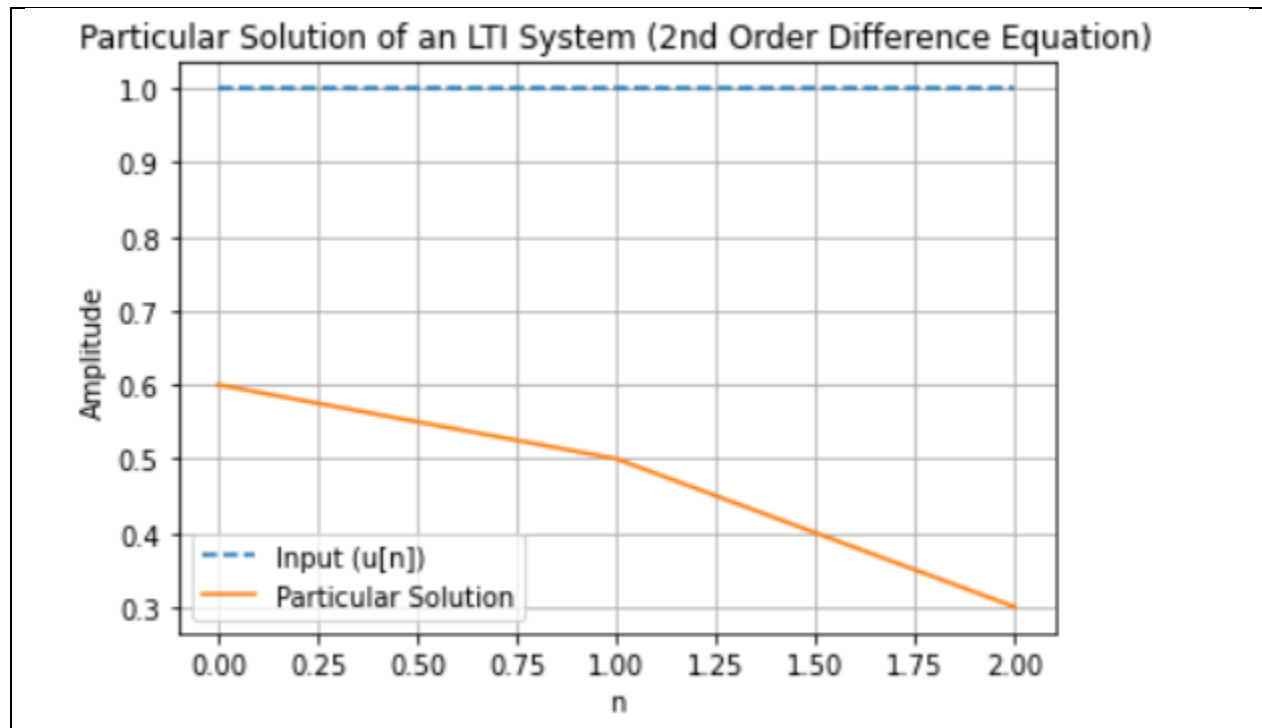
```

def particular_solution(a, b, x):
    Calculate the particular solution of an LTI system.
    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.
    Returns:
    list: Particular solution of the LTI system.
    particular = []
    for n in range(len(x)):
        sum_val = 0
        for i in range(len(b)):
            if n - i >= 0:
                sum_val += b[i] * x[n - i]
        for j in range(1, len(a)):
            if n - j >= 0:
                sum_val -= a[j] * particular[n - j]
        particular.append(sum_val / a[0])
    return particular

a = [1, -2, -1.6]
b = [0.3, 0.2, 0.1]
n = np.arange(0, 10, 1)
x = np.heaviside(n, 1)
particular_sol = particular_solution(a, b, x)
plt.stem(n, x, label="Input (u[n])", basefmt=" ", use_line_collection=True)
plt.stem(n, particular_sol, label="Particular Solution", basefmt=" ", use_line_collection=True,
linefmt="r")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.title('Particular Solution of an LTI System (2nd Order Difference Equation)')
plt.legend()
plt.grid(True)
plt.show()

```

The results (Screenshot)



Lab Task No C:

Solution:

Brief description (3-5 lines)

Zero-State Response:

The zero-state response or forced response is obtained from the sum of homogeneous solution and particular solution and evaluating the constants with zero initial conditions.

Mathematical Equation:

$$y_{zs}(n) = y_h(n) + y_p(n) \quad | \text{ with constants } C_1, C_2, \dots, C_N \text{ evaluated with zero initial conditions}$$

The code

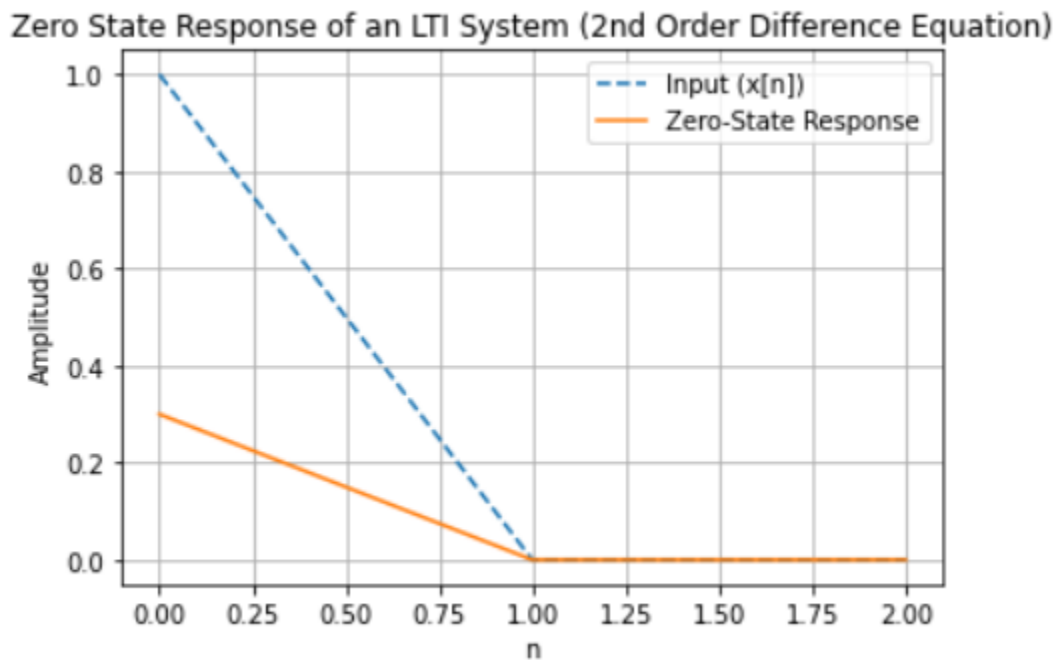
```
# Zero-State Response
import matplotlib.pyplot as plt
def zero_state_response(a, b, x):
    """
    Calculate the zero-state response of an LTI system.
    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.
    Returns:
    list: Zero-state response of the LTI system.
```

```

"""
# Calculate particular solution
particular_solution = [sum(xb * bi for xb, bi in zip(x[n:], b)) for n in range(len(x))]
# Zero homogeneous solution (zero initial conditions)
homogeneous_solution = [0] * len(x)
# Zero-state response
zero_state = [ha + pa for ha, pa in zip(homogeneous_solution, particular_solution)]
return zero_state
# Difference equation coefficients
a = [1, -0.5, -0.3]
b = [0.3, 0.2, 0.1]
n = np.arange(0, 3, 1)
# Input signal (decreasing linear function)
x = np.where(n < 1, 1, 0)
# Calculate zero-state response
zero_state = zero_state_response(a, b, x)
# Plot the input and output
plt.plot(x, label="Input (x[n])", linestyle="--")
plt.plot(zero_state, label="Zero-State Response")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()

```

The results (Screenshot)



Lab Task No D:

Solution:

Brief description (3-5 lines)
Total Response: The overall response of the system to a given input signal, considering both the homogeneous solution and the particular solution. This response can be obtained by adding the zero-input response (homogeneous solution) and the zero-state response (particular solution). The total response is given by sum of zero-input response and zero-state response. Mathematical Equation: $y(n) = y_{zi}(n) + y_{zs}(n)$
The code

```
# Total Response
import numpy as np
import matplotlib.pyplot as plt
def zero_input_response(a, ic):
    Calculate the zero-input response of an LTI system (homogeneous solution).
    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    ic (list): Initial conditions of the system.
    Returns:
    list: Zero-input response of the LTI system.
    """
    zi_response = [sum(ai * ic[m] for m, ai in enumerate(a[1:], 1)) for n in range(len(ic))]
    return zi_response[::-1]
def zero_state_response(a, b, x):
    Calculate the zero-state response of an LTI system (particular solution with zero initial
    conditions).
    Parameters:
    a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
    b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
    x (list): Input signal to the LTI system.
    Returns:
    list: Zero-state response of the LTI system.
    zs_response = [sum(xb * bi for xb, bi in zip(x[n:], b)) for n in range(len(x))]
    return zs_response
def total_response(a, b, x, ic):
    Calculate the total response of an LTI system.
    Parameters:
```

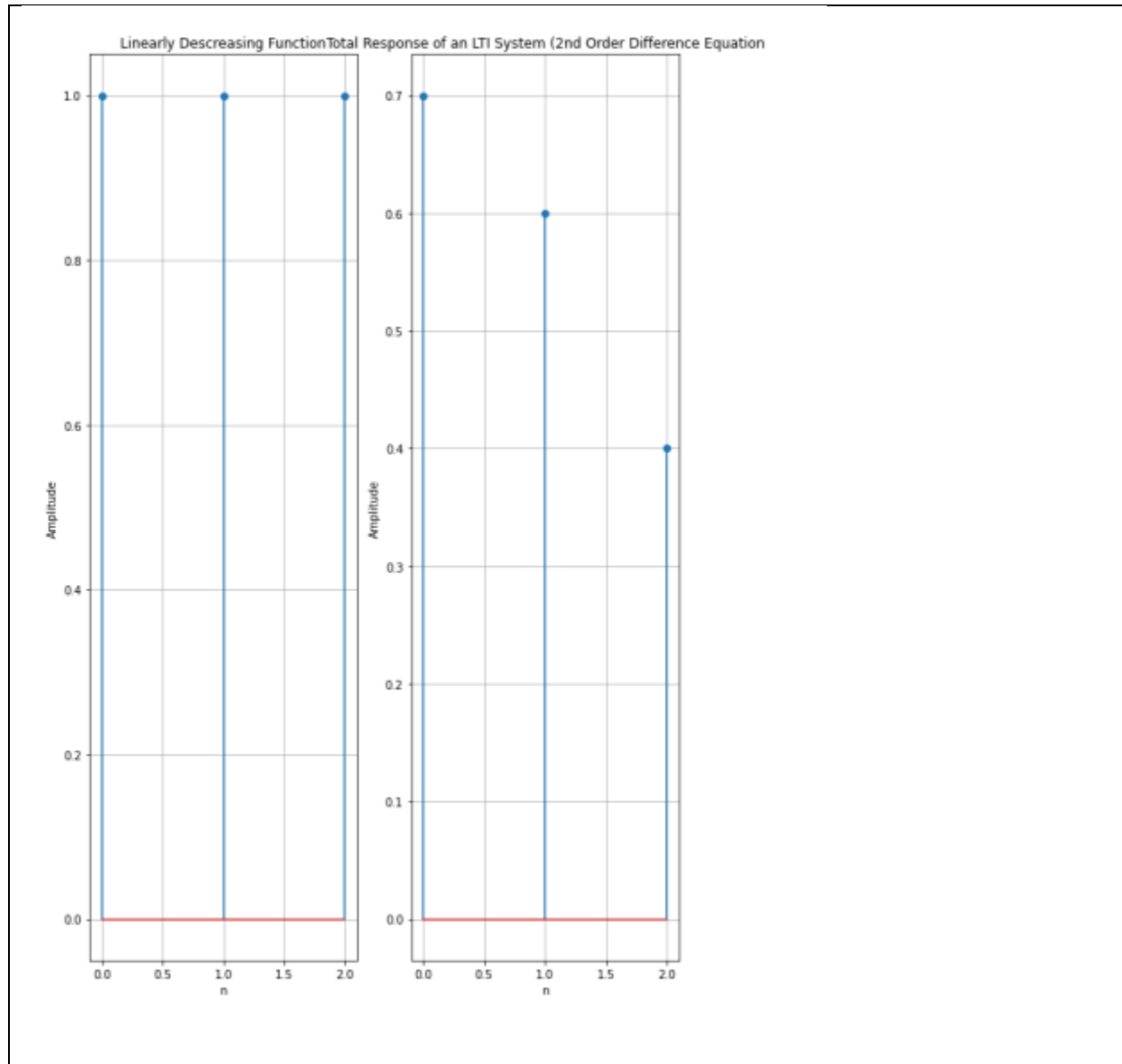
```

a (list): Coefficients of the difference equation (a[0], a[1], ..., a[n]).
b (list): Coefficients of the difference equation (b[0], b[1], ..., b[m]).
x (list): Input signal to the LTI system.
ic (list): Initial conditions of the system.
Returns:
list: Total response of the LTI system.
"""

zero_state = zero_state_response(a, b, x)
zero_input = zero_input_response(a, ic)
return [zs + zi for zs, zi in zip(zero_state, zero_input)]
# Difference equation coefficients
a = [1, -0.5, -0.3]
b = [0.3, 0.2, 0.1]
# Initial conditions
ic = [0, 1, -2]
# Input signal (decreasing linear function)
n = np.arange(0, 3, 1)
x = np.where(n >= 0, 1, 0)
# Calculate total response
total_response_output = total_response(a, b, x, ic)
# Plot the input, zero-input response, zero-state response, and total response
fig, axs = plt.subplots(1, 2, figsize=(8, 12))
# Input signal
axs[0].stem(n, x, label="Input (x[n])")
axs[0].set_xlabel("n")
axs[0].set_ylabel("Amplitude")
axs[0].set_title("Linearly Decreasing Function")
axs[0].grid(True)
# Zero-input response (homogeneous solution)
axs[1].stem(n, total_response_output, label="Total Response")
axs[1].set_xlabel("n")
axs[1].set_ylabel("Amplitude")
axs[1].set_title("Total Response of an LTI System (2nd Order Difference Equation)")
axs[1].grid(True)
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Lab Task No 1(i):

Solution:

Brief description (3-5 lines)

Write Code for all four responses of first order discrete time system governed by the difference equation, $y(n) = -0.8 y(n - 1) + x(n) + 3 x(n - 1)$.

When the input is unit step, and with initial condition a) $y(-1) = 0$ b) $y(-1) = 2.5$.

Zero-Input Response (Homogeneous Solution):

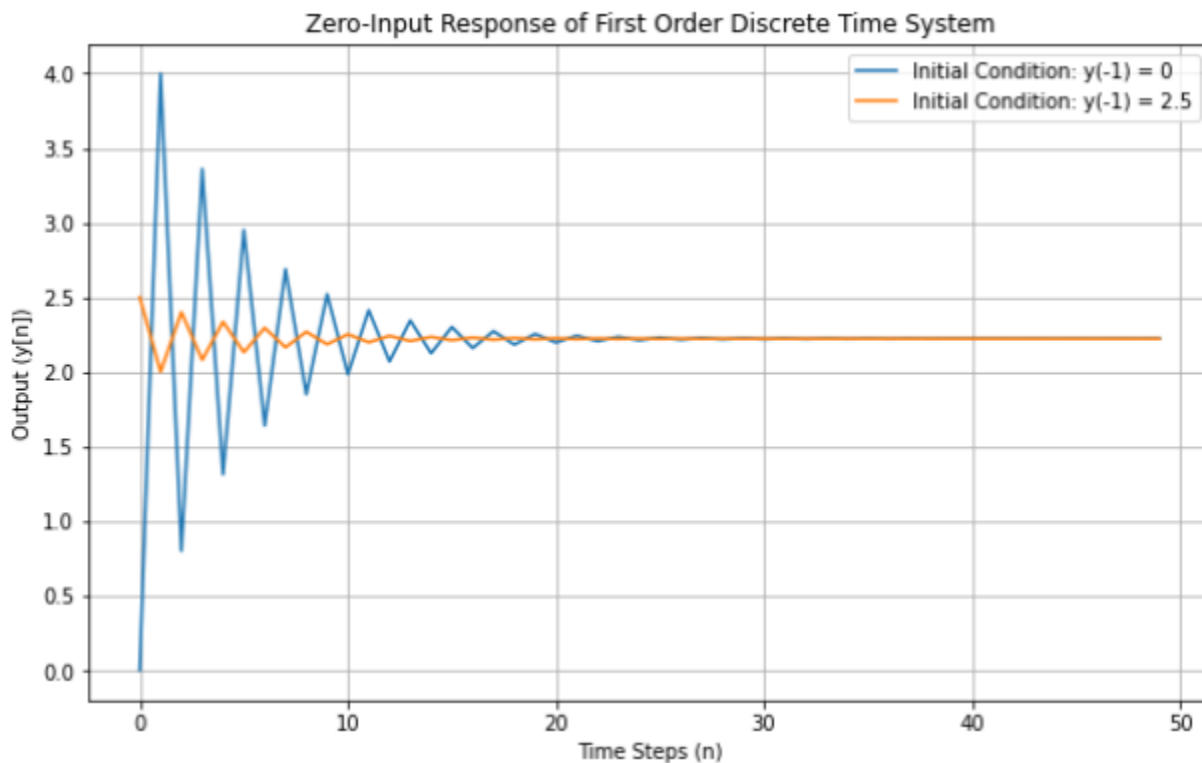
The zero-input response, also known as the homogeneous solution, represents the system's response when no input signal is applied. It is solely determined by the

initial conditions of the system.

The code

```
1 # Zero-Input Response (Homogeneous Solution)
2 import numpy as np
3 import matplotlib.pyplot as plt
4 def difference_equation(y_prev, x_curr, x_prev):
5     return -0.8 * y_prev + x_curr + 3 * x_prev
6 def zero_input_response(n_steps, initial_condition):
7     y = np.zeros(n_steps)
8     y[0] = initial_condition
9     x = np.ones(n_steps) # Unit step input
10    for n in range(1, n_steps):
11        y[n] = difference_equation(y[n-1], x[n], x[n-1])
12    return y
13 n_steps = 50
14 n = np.arange(n_steps)
15 y_a = zero_input_response(n_steps, initial_condition=0)
16 y_b = zero_input_response(n_steps, initial_condition=2.5)
17 plt.figure(figsize=(10, 6))
18 plt.plot(n, y_a, label='Initial Condition:  $y(-1) = 0$ ')
19 plt.plot(n, y_b, label='Initial Condition:  $y(-1) = 2.5$ ')
20 plt.title('Zero-Input Response of First Order Discrete Time System')
21 plt.xlabel('Time Steps (n)')
22 plt.ylabel('Output (y[n])')
23 plt.legend()
24 plt.grid(True)
25 plt.show()
```

The results (Screenshot)



Lab Home Task No 1(ii):

Solution:

Brief description (3-5 lines)

Write Code for all four responses of first order discrete time system governed by the difference equation, $y(n) = -0.8 y(n - 1) + x(n) + 3 x(n - 1)$.

When the input is unit step, and with initial condition a) $y(-1) = 0$ b) $y(-1) = 2.5$.

Particular Solution:

The particular solution represents the system's response to a specific input signal, ignoring the initial conditions. It captures the transient behavior of the system caused by the input signal.

The code

```
1  # Particular Solution
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def first_order_system(input_signal, initial_condition):
5      a1 = -0.8
6      b0 = 1
7      b1 = 3
8      N = len(input_signal)
9      output_signal = np.zeros(N)
10     output_signal[0] = initial_condition
11     for n in range(1, N):
12         output_signal[n] = b0 * input_signal[n] + b1 * input_signal[n - 1] + a1 * output_signal[n - 1]
13     return output_signal
14 N = 50
15 input_signal = np.ones(N)
16 initial_condition_a = 0
17 output_signal_a = first_order_system(input_signal, initial_condition_a)
18 initial_condition_b = 2.5
19 output_signal_b = first_order_system(input_signal, initial_condition_b)
20 plt.figure(figsize=(10, 6))
21 plt.subplot(2, 1, 1)
22 plt.stem(output_signal_a, linefmt='b-', markerfmt='bo', basefmt='k-', label='y(-1) = 0')
23 plt.title('Response of First Order System (Unit Step Input)')
24 plt.xlabel('n')
25 plt.ylabel('y(n)')
26 plt.grid(True)
27 plt.legend()
28 plt.subplot(2, 1, 2)
29 plt.stem(output_signal_b, linefmt='r-', markerfmt='ro', basefmt='k-', label='y(-1) = 2.5')
30 plt.xlabel('n')
31 plt.ylabel('y(n)')
32 plt.grid(True)
33 plt.legend()
34 plt.tight_layout()
35 plt.show()
```

The results (Screenshot)



Lab Task No 1(iii):

Solution:

Brief description (3-5 lines)

Write Code for all four responses of first order discrete time system governed by the difference equation, $y(n) = -0.8 y(n - 1) + x(n) + 3 x(n - 1)$.

When the input is unit step, and with initial condition a) $y(-1) = 0$ b) $y(-1) = 2.5$.

Zero-State Response:

The zero-state response or forced response is obtained from the sum of homogeneous solution and particular solution and evaluating the constants with zero initial conditions.

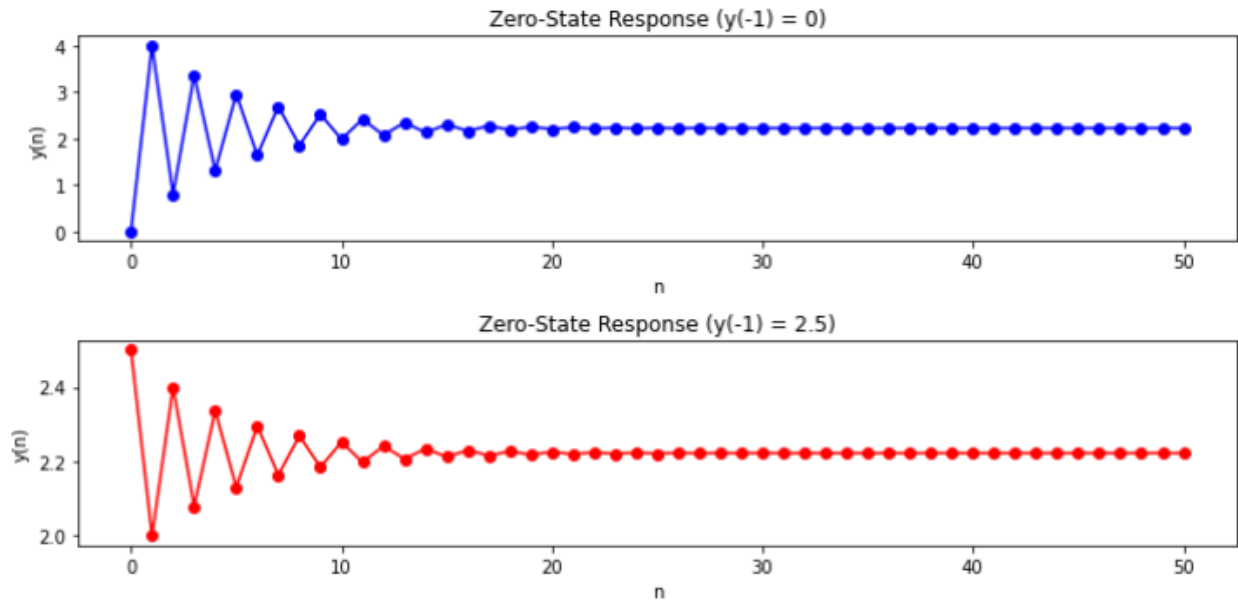
The code

```

1  # Zero-State Response
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def difference_equation(y_minus_1, N):
5      a1 = -0.8
6      b0 = 1
7      b1 = 3
8      x = np.ones(N+1) # Unit step input
9      y = np.zeros(N+1) # Output
10     y[0] = y_minus_1
11     for n in range(1, N+1):
12         y[n] = b0*x[n] + b1*x[n-1] + a1*y[n-1]
13     return y
14     N = 50
15     y_zero_init = difference_equation(0, N)
16     y_nonzero_init = difference_equation(2.5, N)
17     plt.figure(figsize=(10, 5))
18     plt.subplot(2, 1, 1)
19     plt.plot(np.arange(N+1), y_zero_init, marker='o', linestyle='-', color='b')
20     plt.title('Zero-State Response (y(-1) = 0)')
21     plt.xlabel('n')
22     plt.ylabel('y(n)')
23     plt.subplot(2, 1, 2)
24     plt.plot(np.arange(N+1), y_nonzero_init, marker='o', linestyle='-', color='r')
25     plt.title('Zero-State Response (y(-1) = 2.5)')
26     plt.xlabel('n')
27     plt.ylabel('y(n)')
28     plt.tight_layout()
29     plt.show()

```

The results (Screenshot)



Lab Home Task No 1(iv):

Solution:

Brief description (3-5 lines)

Write Code for all four responses of first order discrete time system governed by the difference equation, $y(n) = -0.8 y(n - 1) + x(n) + 3 x(n - 1)$.

When the input is unit step, and with initial condition a) $y(-1) = 0$ b) $y(-1) = 2.5$.

Total Response:

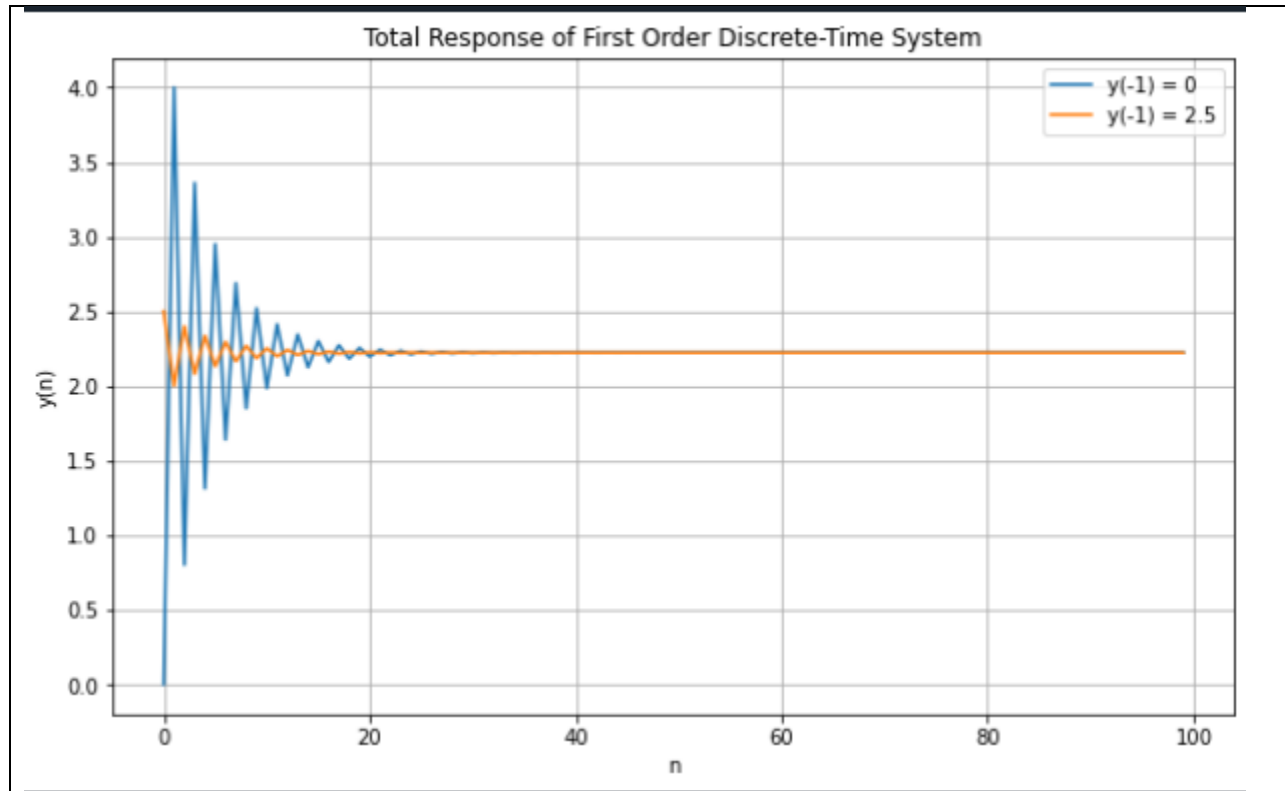
The overall response of the system to a given input signal, considering both the homogeneous solution and the particular solution. This response can be obtained by adding the zero-input response (homogeneous solution) and the zero-state response (particular solution).

The total response is given by sum of zero-input response and zero-state response.

The code

```
1  # Total Response |
2  import numpy as np
3  import matplotlib.pyplot as plt
4  def system_response(x, y_prev, y_prev_init):
5      N = len(x)
6      y = np.zeros(N)
7      y[0] = y_prev_init
8      for n in range(1, N):
9          y[n] = -0.8 * y[n - 1] + x[n] + 3 * x[n - 1]
10     return y
11     N = 100
12     x = np.ones(N)
13     y_init_1 = 0
14     y_init_2 = 2.5
15     y_response_1 = system_response(x, 0, y_init_1)
16     y_response_2 = system_response(x, 0, y_init_2)
17     plt.figure(figsize=(10, 6))
18     plt.plot(y_response_1, label=f'y(-1) = {y_init_1}')
19     plt.plot(y_response_2, label=f'y(-1) = {y_init_2}')
20     plt.title('Total Response of First Order Discrete-Time System')
21     plt.xlabel('n')
22     plt.ylabel('y(n)')
23     plt.legend()
24     plt.grid(True)
25     plt.show()
```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

Determine the response $y(n)$, $n > 0$ of the system described by the second order difference equation, $y(n) - 0.2 y(n - 1) - 0.03 y(n - 2) = x(n) + 0.4 x(n - 1)$.

When the input signal is, $x(n) = 0.2n u(n)$ and with initial conditions $y(-2) = 0$, $y(-1) = 0.5$.

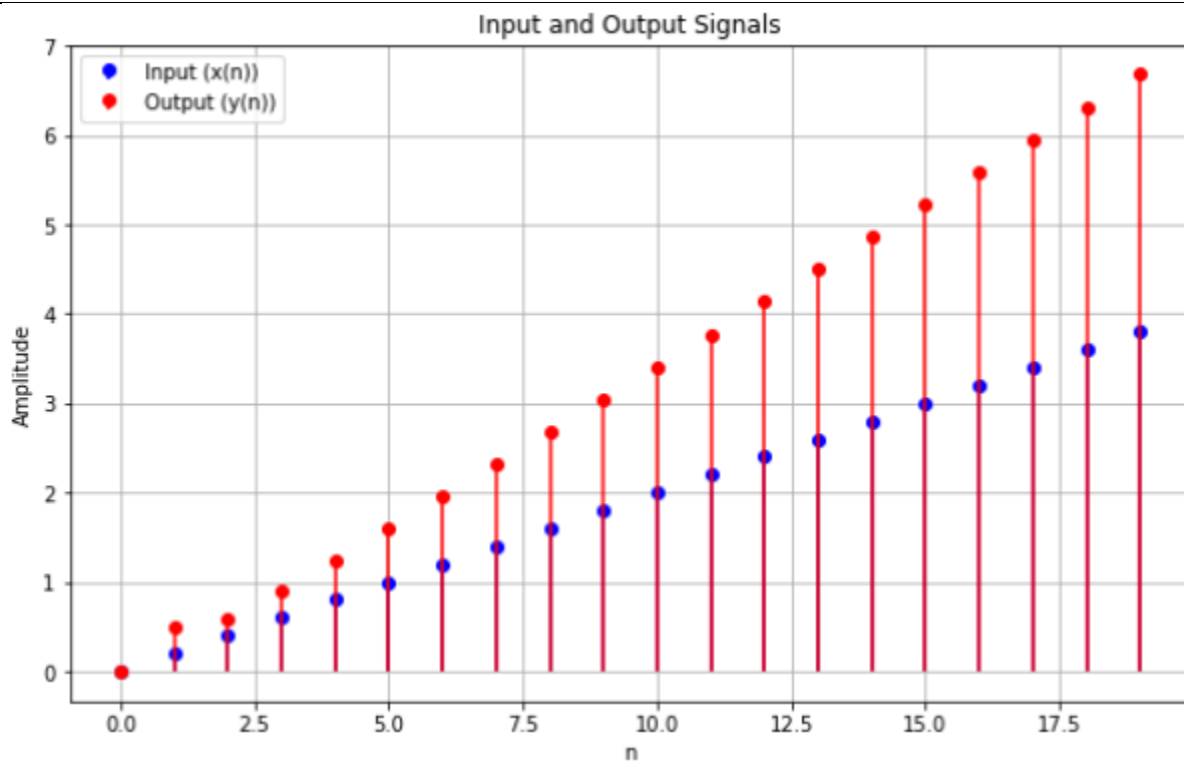
The code

```

1  # second order difference equation
2  import numpy as np
3  import matplotlib.pyplot as plt
4  a = [1, -0.2, -0.03]
5  b = [1, 0.4]
6  def input_signal(n):
7      return 0.2 * n * (n >= 0)
8  def system_response(x, a, b, y0, y1):
9      y = np.zeros_like(x)
10     y[0] = y0
11     y[1] = y1
12     for n in range(2, len(x)):
13         y[n] = -a[1]*y[n-1] - a[2]*y[n-2] + b[0]*x[n] + b[1]*x[n-1]
14     return y
15 n_values = np.arange(0, 20)
16 x = input_signal(n_values)
17 y = system_response(x, a, b, 0, 0.5)
18 plt.figure(figsize=(10, 6))
19 plt.stem(n_values, x, linefmt='b-', markerfmt='bo', basefmt=' ')
20 plt.stem(n_values, y, linefmt='r-', markerfmt='ro', basefmt=' ')
21 plt.xlabel('n')
22 plt.ylabel('Amplitude')
23 plt.title('Input and Output Signals')
24 plt.legend(['Input (x(n))', 'Output (y(n))'])
25 plt.grid(True)
26 plt.show()

```

The results (Screenshot)



Conclusion:

In conclusion, we had studied the four responses of LTI Discrete Time System in Time Domain. The four response that is Zero-Input Response (Homogeneous Solution), Particular Solution, Zero-State Response, Total Response and second order difference equation. There working ,functions in digital system and impact in our daily life.